第10堂｜實作：AI 智慧驅動視覺化簡報生成

學習目標

完成本堂課後，學員將能夠：

- 使用 AI 將長篇文稿或大綱自動轉化為結構化的簡報邏輯
- 使用 python-pptx 套件以程式方式生成 PowerPoint 簡報
- 自動化套用一致的設計風格至所有投影片
- 建立從「文字輸入」到「完整簡報輸出」的端到端自動化流程

核心概念說明

1. 為什麼簡報製作是 AI 自動化的理想場景？

製作一份簡報通常需要：

1. 閱讀大量資料 → **AI 最擅長的摘要與結構化**
2. 決定每張投影片的內容邏輯 → **AI 的架構規劃能力**
3. 確保視覺風格一致 → **程式操控 PPT 的強項**
4. 反覆調整格式和排版 → **程式比人工快、精準**

簡報製作因此是「AI 語意能力 + 程式自動化」結合效益最大的場景之一。

---

2. AI 語意架構化：從長文到投影片結構

核心任務： 將一份冗長的文稿或大綱，轉化為「每張投影片該說什麼」的清單。

好的投影片結構原則：

原則	說明
金字塔原則	結論先行，再提供支撐細節
一頁一訊息	每張投影片只傳達一個核心觀點
3的法則	能分成三點的，就分三點（容易記憶）
故事弧線	問題 → 影響 → 解決方案 → 行動

AI 架構化的 Prompt 策略：

```
輸入：一篇 1000 字的市場分析報告

需要 AI 完成的任務：
1. 識別報告的核心論點（1-2句）
2. 拆解為 8-12 張投影片的架構
3. 每張投影片提供：
```

```
    - 投影片標題（8字以內，有力、精準）
    - 投影片類型（標題頁、大綱、內容、數據、結論）
    - 主要內容（3-5個條列重點或1段文字）
    - 建議的視覺輔助（圖表類型、圖示建議）
4. 以 JSON 格式輸出，方便程式讀取
```

---

3. PowerPoint 自動化：python-pptx

python-pptx 是一個 Python 套件，可以用程式碼建立、修改 PowerPoint 檔案（.pptx）。

投影片的組成元素：

元素	說明
Slide（投影片）	每一頁投影片
Slide Layout（版面配置）	預設的排版模板（標題頁、內容頁等）
Placeholder（接受框）	版面配置中的文字或圖片位置
TextFrame（文字框）	包含文字的容器
Paragraph（段落）	文字框內的一個段落
Run（文字段）	段落中相同格式的一段文字
Shape（形狀）	矩形、圓形、線條等
Chart（圖表）	長條圖、折線圖、圓餅圖

設計一致性的實現方式：

```
一致設計的要素：
    主題色彩：定義 3-5 個主色和輔助色
    字型規範：標題字體 / 內文字體 / 大小規格
    版面規則：每張投影片左側留白多少、標題位置固定

在 python-pptx 中：
    使用 Presentation Theme 套用整體樣式
    使用 Slide Master 統一管理母片設計
    每張投影片繼承相同的基礎設計，再加入個別內容
```

---

4. 投影片類型與對應視覺設計

不同類型的投影片需要不同的版面設計：

投影片類型	適合的版面	主要元素
封面頁	全版面視覺 + 大標題	主標題、副標題、日期、機構

投影片類型	適合的版面	主要元素
大綱頁	條列式清單	本次簡報的章節架構
內容頁（文字）	標題 + 條列點	3-5個重點（每點一行最佳）
數據頁	標題 + 圖表	一個清楚的圖表 + 簡短說明
比較頁	左右分欄	兩個選項並排比較
過渡頁	大字、少內容	章節標題，告訴觀眾接下來要講什麼
結論頁	重點摘要	3-5個最重要的帶走訊息
行動號召頁	醒目視覺	明確的下一步行動

---

5. 完整的簡報生成流程

```
使用者輸入：
    原始文稿 / 大綱 / 關鍵字
    簡報主題與目的
    目標受眾
    風格偏好（正式/活潑/科技感）
    投影片數量限制
        │
        ▼ 第一步：AI 架構規劃
    AI 輸出 JSON 格式的投影片架構清單
        │
        ▼ 第二步：人工確認（可選）
    使用者確認或調整架構
        │
        ▼ 第三步：程式生成 PPT
    python-pptx 讀取 JSON，逐張生成投影片
        │
        ▼ 第四步：套用設計風格
    套用指定的色彩、字型、版面規則
        │
        ▼ 輸出
    完整的 .pptx 簡報檔案（可在 PowerPoint 中進一步編輯）
```

實作步驟（逐步教學）

實作一：AI 架構規劃模組

步驟 1：安裝套件

```
pip install python-pptx
```

步驟 2：向 GitHub Copilot 描述架構規劃函式

```
請幫我建立一個「投影片架構規劃」函式：
1. 輸入：原始文稿（字串）、簡報主題、目標張數（預設 10）
2. 呼叫 OpenAI API，讓 AI 設計投影片架構
3. AI 回傳的格式必須是合法的 JSON，結構如下：
   {
     "title": "整份簡報的標題",
     "slides": [
       {
         "index": 1,
         "type": "cover/outline/content/data/transition/conclusion",
         "title": "投影片標題",
         "points": ["重點1", "重點2", "重點3"],
         "notes": "演講者備注"
       },
       ...
     ]
   }
4. 如果 AI 回傳不是合法 JSON，要能重試（最多 3 次）
```

步驟 3：測試架構規劃

準備一段 500-800 字的文稿，執行架構規劃函式， 觀察 AI 規劃的投影片架構是否合理。

實作二：PowerPoint 自動生成

步驟 1：設計投影片版面函式

```
請幫我用 python-pptx 建立一個 PPT 生成函式，以下是要求：
1. 一個主要函式：generate_presentation(slides_json, output_path)
2. 支援的投影片類型（以 slides_json 中的 type 欄位判斷）：
   - "cover"：封面（大標題置中，副標題，顯示今天日期）
   - "content"：內容頁（左上角標題，下方 3-5 個條列重點）
   - "transition"：過渡頁（章節標題置中，大字，背景色不同）
   - "conclusion"：結論頁（標題 + 重點摘要）
3. 所有投影片使用統一的設計規範：
   - 標題字型：微軟正黑體，32pt，黑色
   - 內文字型：微軟正黑體，20pt，深灰色
   - 主色調：深藍色 (#1A3A5C)，過渡頁用主色背景
   - 頁碼顯示在右下角（封面除外）
```

步驟 2：加入圖表功能（資料說明頁）

```
請幫我加入圖表支援：
如果投影片類型是 "data" 且 JSON 中有 chart_data 欄位：
- 建立一個長條圖或折線圖（依 chart_type 欄位決定）
- 將圖表嵌入投影片右側
- 左側顯示 2-3 個重點說明
```

實作三：整合完整流程

步驟 1：建立主程式

```
請幫我建立一個主程式，整合以上所有功能：
1. 讀取使用者輸入（從命令列或 JSON 設定檔）：
   - 輸入文稿路徑
   - 簡報主題
   - 投影片數量
   - 輸出 PPT 路徑
2. 呼叫架構規劃函式，取得 JSON 架構
3. 顯示規劃好的架構，詢問使用者是否確認（y/n）
4. 若確認，呼叫 PPT 生成函式
5. 生成完成後顯示成功訊息和輸出路徑
```

程式碼範例（虛擬碼）

AI 投影片架構規劃

```
函式 規劃投影片架構(文稿內容, 簡報主題, 目標張數=10):
    提示詞 = f"""
    你是一位專業的簡報設計顧問。
    請將以下文稿設計成 {目標張數} 張左右的投影片架構。

    簡報主題：{簡報主題}
    文稿內容：
    {文稿內容}

    請以 JSON 格式回傳，結構如下：
    {{
        "title": "簡報總標題",
        "slides": [
            {{
                "index": 1,
                "type": "cover",
                "title": "...",
                "subtitle": "...",
                "points": [],
                "notes": "演講者備注"
            }},
            ...
        ]
    }}

    設計原則：
    - 第一張必須是封面（cover 類型）
    - 最後一張必須是結論或行動號召
```

```
    - 每張投影片傳達一個清楚的重點
    - 純文字的 points 每個條列不超過 20 字
    - 只回傳 JSON，不要有其他說明文字
    """

    對 重試次數 從 1 到 3:
        嘗試:
            AI回應 = 呼叫LLM(提示詞)
            結果 = 解析JSON(AI回應)
            回傳 結果
        如果發生 JSON解析錯誤:
            顯示 f"第 {重試次數} 次 JSON 解析失敗，重試中..."

    拋出例外("無法取得合法的架構 JSON")
```

PowerPoint 投影片生成

```
載入 pptx 套件
載入 pptx.util (Inches, Pt, Emu)
載入 pptx.dml.color (RGBColor)

常數:
    主色   = RGBColor(0x1A, 0x3A, 0x5C)  # 深藍色
    深灰色  = RGBColor(0x33, 0x33, 0x33)
    白色   = RGBColor(0xFF, 0xFF, 0xFF)
    標題字體 = "微軟正黑體"
    內文字體 = "微軟正黑體"

函式 生成投影片(架構JSON, 輸出路徑):
    簡報 = 建立新的Presentation()

    對 每張投影片資訊 在 架構JSON["slides"] 中:
        類型 = 投影片資訊["type"]
        新投影片 = 簡報.加入新投影片()

        如果 類型 == "cover":
            新投影片.背景色 = 主色
            加入文字框(新投影片, 投影片資訊["title"],
                       位置=置中, 字體=白色, 大小=40pt, 粗體=真)
            如果 有副標題:
                加入文字框(新投影片, 投影片資訊["subtitle"],
                           位置=副標題位置, 字體=白色半透明, 大小=24pt)

        否則如果 類型 == "content":
            加入標題(新投影片, 投影片資訊["title"], 顏色=主色)
            對 每個重點 在 投影片資訊["points"] 中:
                加入條列式文字(新投影片, 重點, 縮排=1)

        否則如果 類型 == "transition":
            新投影片.背景色 = 主色
            加入置中大字(新投影片, 投影片資訊["title"], 顏色=白色, 大小=36pt)
```

```
        # 加入頁碼（封面除外）
        如果 類型 != "cover":
            加入頁碼(新投影片, 投影片資訊["index"])

    簡報.儲存(輸出路徑)
    顯示 f"簡報已儲存：{輸出路徑}"
```

練習題 / 作業

課堂練習

練習 1：投影片架構邏輯

給你以下一份短文，試著手動規劃成 5 張投影片的架構，再看看 AI 會怎麼規劃，比較差異：

> 台灣的中小企業面臨數位轉型的挑戰。根據調查，僅有 30% 的中小企業已完成基礎數位化。主要障礙包括：預算不足、人才短缺、對新技術不熟悉。然而，那些已完成轉型的企業平均提升了 25% 的生產力。解決方案包括：政府補助計畫、雲端工具導入、員工培訓計畫。未來三年內，不進行數位轉型的企業將面臨嚴峻的競爭劣勢。

練習 2：設計風格決策

思考以下場景，各需要什麼設計風格的簡報：

場景	受眾	適合的風格（正式/活潑/科技感）	色調建議
向投資人的募資簡報	投資人		
員工教育訓練	內部員工		
產品發表會	潛在客戶		
學術研究成果報告	學術圈		

課後作業

1. **完整流程測試**：準備一份 600-1000 字的報告或文章（可以是工作相關的文件），執行本堂課建立的完整流程，生成一份 PPT，並評估輸出品質。
2. **設計改進**：觀察生成的 PPT，找出 2-3 個設計上的不足，思考如何修改程式碼來改善。

延伸閱讀

PowerPoint 自動化：

- python-pptx 官方文件 — https://python-pptx.readthedocs.io/
- python-pptx 圖表教學 — 文件中的 Charts 章節

簡報設計原則：

- 《簡報聖經》by Garr Reynolds — 好簡報的設計原則
- 《金字塔原理》by Barbara Minto — 結構化思考與表達的經典
- TED 簡報技巧 — 搜尋「TED talk presentation tips」

AI 圖像生成（擴展閱讀）：

- DALL-E 3 API — 可整合至流程，自動生成簡報配圖
- Stable Diffusion — 開源圖像生成，可本地運行